ADOPT-01: Observability Maturity Model

> **Series:** ADOPT — Observability Adoption & Maturity | **Notebook:** 1 of 6 | **Created:** March 2026 | **Last Updated:** 10/21/2026

Overview

Observability maturity is not a binary state — organizations progress through distinct levels as their practices, tooling, and culture evolve. This notebook introduces a five-level maturity model for Dynatrace-powered observability, provides assessment criteria for each level, and demonstrates how to use DQL queries to gauge your current position. Understanding where you stand today is the first step toward building a roadmap for improvement.

> **This is a community framework, not a Dynatrace-published model.** The five levels, their indicators, and the feature mapping below are a planning aid assembled from community practice. Use them to structure a conversation about where to invest next, not as a benchmark Dynatrace measures you against.

Platform-Evolution Markers Worth Tracking

Three platform changes give a maturity program concrete, measurable milestones. Each maps onto the levels in this notebook:

1. **Primary Grail fields and tags enriched at the source** (OneAgent 1.333+) — OneAgent can stamp `dt.security_context`, `dt.cost.costcenter`, `dt.cost.product` and `primary_tags.*` onto every signal it sends. **Level 3** programs surface these fields in dashboards; **Level 4** programs drive bucket routing, access control and cost allocation from them; **Level 5** programs set them at deployment time so no host or process ships without them. Configuration lives in ORGNZ and FAQ-02.

2. **Extensions Framework 1.0 retirement** — Extensions Framework 1.0 reached end of support on 2025-03-31 (its Python 3.8 variant on 2024-10-31). JMX and PMI extensions on Framework 1.0 were carried past that date as deprecated, and now have their own end-of-support date: **July 1, 2027**. Count the Framework 1.0 extensions still running and track that number to zero in the quarterly health review (ADOPT-02).
3. **OneAgent and OpenTelemetry-injector coexistence** (K8S-11 § 2a) — clusters that run both injectors need an explicit per-namespace decision. A **Level 3** program documents which namespaces each injector owns; a **Level 5** program enforces that split with an admission or policy gate.

Add these to the platform-health checklist in ADOPT-02 and the success metrics in ADOPT-03.

Table of Contents

1. The Five Maturity Levels
2. Level 1 — Reactive Monitoring
3. Level 2 — Proactive Monitoring
4. Level 3 — Data-Driven Observability
5. Level 4 — Predictive Observability
6. Level 5 — Autonomous Operations
7. Assessing Your Current Level
8. Mapping Dynatrace Features to Maturity Levels
9. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS (Grail). The DQL in this notebook does not apply to Dynatrace Managed, which has no Grail — the DTM series (Dynatrace Managed) covers Managed.

Requirement	Details
Permissions	`storage:entities:read`, `storage:smartscape:read`, `storage:logs:read`, `storage:spans:read`, `storage:events:read`
Data	At least 24 hours of ingested monitoring data
Audience	Platform engineers, SREs, engineering managers, and leadership stakeholders

1. The Five Maturity Levels

The observability maturity model defines five progressive levels. Each level builds on the previous one, adding capabilities and shifting organizational behavior.

Level	Name	Characteristics	Key Indicator
1	Reactive	Alert-driven, manual troubleshooting, siloed tools	"We find out about problems from users"
2	Proactive	Threshold-based alerts, basic dashboards, some automation	"We detect problems before users report them"
3	Data-Driven	Centralized observability, SLOs defined, DQL-powered analysis	"We make decisions based on telemetry data"
4	Predictive	AI-driven anomaly detection, forecasting, capacity planning	"We predict problems before they occur"
5	Autonomous	Self-healing, automated remediation, closed-loop operations	"The platform resolves problems without human intervention"

Most organizations operate between Level 1 and Level 3. The goal is not necessarily to reach Level 5 everywhere, but to deliberately choose the appropriate level for each service based on business criticality.

2. Level 1 — Reactive Monitoring

Characteristics

- Monitoring is an afterthought, added post-deployment
- Alerts are noisy and frequently ignored
- Troubleshooting relies on SSH-ing into hosts and reading log files
- No standardized observability tooling across teams
- Mean Time to Detect (MTTD) is measured in hours, often reported by end users

Assessment Criteria

Criterion	Level 1 Indicator
Agent coverage	< 50% of hosts instrumented
Data sources	Only infrastructure metrics
Alerting	Static thresholds or none
Dashboards	Ad-hoc, per-team, no standards
Incident process	Manual, tribal knowledge

Dynatrace Features at This Level

- OneAgent deployment on critical hosts
- Basic infrastructure monitoring
- Default Dynatrace Intelligence alerting (often not yet configured)

3. Level 2 — Proactive Monitoring

Characteristics

- Monitoring is part of the deployment checklist
- Threshold-based alerts detect common failure modes
- Centralized dashboards exist but may not be maintained

- Teams respond to problems before user impact escalates
- MTTD is measured in minutes

Assessment Criteria

Criterion	Level 2 Indicator
Agent coverage	50-80% of hosts instrumented
Data sources	Infrastructure + application metrics
Alerting	Threshold-based, some Dynatrace Intelligence
Dashboards	Standardized templates for key services
Incident process	Defined runbooks for top 10 issues

Dynatrace Features at This Level

- Full-stack OneAgent deployment
- Dynatrace Intelligence problem detection enabled
- Custom anomaly detectors for known failure modes (ALERT-02)
- Basic synthetic monitoring for key URLs

4. Level 3 — Data-Driven Observability

Characteristics

- Observability data drives architectural and operational decisions
- SLOs are defined and tracked for critical services
- Teams use DQL and Grail for ad-hoc investigation
- Log, trace, and metric data are correlated
- Observability is treated as a platform service

Assessment Criteria

Criterion	Level 3 Indicator
Agent coverage	> 80% of hosts and services instrumented
Data sources	Infrastructure + APM + logs + traces + RUM
Alerting	SLO-based alerting, reduced noise
Dashboards	Business-context dashboards, executive views
Incident process	Data-driven RCA, blameless postmortems

Dynatrace Features at This Level

- Grail data lakehouse with DQL queries
- OpenPipeline for log routing and enrichment
- SLO definitions, with burn-rate alerting built as described in SLO-04
- Distributed tracing with span analytics
- Real User Monitoring (RUM) and Session Replay

5. Level 4 — Predictive Observability

Characteristics

- AI-driven anomaly detection replaces static thresholds
- Capacity planning uses forecasting models
- Problems are identified before they cause user impact
- Teams focus on trends and patterns rather than individual alerts

Assessment Criteria

Criterion	Level 4 Indicator
Agent coverage	> 95% with auto-discovery
Data sources	All telemetry types + business events

Criterion	Level 4 Indicator
Alerting	AI-driven, predictive, low false-positive rate
Dashboards	Predictive views, forecasting charts
Incident process	Automated triage and enrichment

Dynatrace Features at This Level

- Dynatrace Intelligence with custom anomaly detection
- Metric forecasting and trend analysis
- Business event correlation
- Ownership and team assignment for entities

6. Level 5 — Autonomous Operations

Characteristics

- Self-healing systems automatically remediate known issues
- Workflows trigger automated responses to problems
- Human intervention is the exception, not the norm
- Continuous optimization is automated

Assessment Criteria

Criterion	Level 5 Indicator
Agent coverage	100% with automated rollout
Data sources	Full telemetry + external integrations
Alerting	Event-driven automation, minimal manual alerts
Dashboards	Real-time executive cockpits
Incident process	Closed-loop: detect → diagnose → remediate → verify

Dynatrace Features at This Level

- Dynatrace Workflows for automated remediation
- Event-driven workflow triggers (problems, events, schedules)
- Configuration as code (Monaco or Terraform) with a drift check — Terraform's `plan` reports drift natively; with Monaco it is a scheduled download-and-diff (AUTOM)
- Full API-driven operations

7. Assessing Your Current Level

The following DQL queries help you assess where your organization stands today. Each query targets a key indicator of maturity.

7.1 Monitored Entity Coverage

Understanding how many entities Dynatrace is monitoring gives a baseline for agent deployment coverage.

```
// Count monitored entities seen in the last 24 hours, by type
fetch dt.entity.host, from:-24h
| summarize host_count = count()
| append [fetch dt.entity.service, from:-24h | summarize service_count =
count()]
| append [fetch dt.entity.process_group, from:-24h | summarize
process_group_count = count()]
| append [smartscapeNodes "FRONTEND", from:-24h | summarize frontend_count =
count()]

// Frontends (web and mobile apps) are counted on Smartscape: on a tenant
using the current RUM,
// classic dt.entity.application returned 0 while smartscapeNodes "FRONTEND"
returned 34
// (validated 10/01/2026). Hosts, services and process groups stay on the
classic surface here
// because this is an inventory, and FAQ-16 explains why the two surfaces can
disagree on counts.
// Every subquery carries from:-24h so all four counts cover the same window.
```

7.2 Data Ingestion Diversity

Mature organizations ingest multiple telemetry types. This query checks which data sources are actively flowing into Grail.

```
// Check log ingestion volume over the last 24 hours
fetch logs, from:-24h
| summarize {log_count = count(), distinct_sources =
countDistinct(log.source)}
```

```
// Check span ingestion to verify distributed tracing is active
fetch spans, from:-24h
| summarize {span_count = count(), distinct_services =
countDistinct(dt.entity.service)}
```

7.3 Dynatrace Intelligence Problem Detection Activity

Active Dynatrace Intelligence problem detection is a key indicator of Level 2+ maturity. If few or no problems are being detected, it may indicate insufficient instrumentation or misconfigured alerting.

```
// Count detected problems in the last 7 days by status
fetch dt.davis.problems, from:-7d
| summarize problem_count = count(), by:{event.status}
| sort problem_count desc
```

7.4 Alerting Quality Check

Two signals separate a noisy problem feed from a useful one, and both can be read straight from problem records:

- **Short-lived problems** — problems that close within a few minutes usually mean a detector that is too sensitive, or a transient nobody needs to be woken for.
- **Problems raised during maintenance** — if a large share of problems open inside maintenance windows, the windows are doing the suppression work that detection should not need.

Earlier versions of this notebook measured noise with `dt.davis.is_frequent_event`. Do not use that flag for this: frequent issue detection is being phased out on the latest

Dynatrace platform, and on the validation tenant it was set on **0 of 15,227 problems** in 30 days — a "noise ratio" built on it reads near zero however noisy alerting actually is.

```
// Alert-quality signals over 7 days: short-lived problems and problems raised
during maintenance
fetch dt.davis.problems, from:-7d
| filter dt.davis.is_duplicate == false
| fieldsAdd duration_min = resolved_problem_duration / 1m
| summarize {
    problems = count(),
    closed_within_5min = countIf(duration_min &lt; 5),
    during_maintenance = countIf(maintenance.is_under_maintenance == true)
  }
| fieldsAdd short_lived_pct = round(toDouble(closed_within_5min) /
toDouble(problems) * 100, decimals: 1)
| fieldsAdd maintenance_pct = round(toDouble(during_maintenance) /
toDouble(problems) * 100, decimals: 1)
```

> **Interpreting the result.** Dynatrace does not publish a healthy value for either percentage, so treat them as trends rather than grades. On the validation tenant (10/01/2026) the query returned 20.8% short-lived and 34.2% during maintenance. In community practice a short-lived share that keeps rising, or a maintenance share above roughly a third, are both signs that problems reach people who do not need to act on them. ADOPT-03 § 5 breaks the short-lived share down by problem type, which is where the fix usually is.

8. Mapping Dynatrace Features to Maturity Levels

Use this table to identify which Dynatrace capabilities to adopt next based on your target maturity level. The level assignments are community guidance, not a product requirement — any feature can be adopted earlier.

Feature	L1	L2	L3	L4	L5	Series
OneAgent (Infrastructure)	X	X	X	X	X	ONBRD
OneAgent (Full-Stack)		X	X	X	X	ONBRD, ADOPT-06

Feature	L1	L2	L3	L4	L5	Series
Kubernetes monitoring		X	X	X	X	K8S
Dynatrace Intelligence problem detection		X	X	X	X	AIOPS
Synthetic Monitoring		X	X	X	X	SYNTH
Log Management (Grail)			X	X	X	OPLOGS
Distributed Tracing (Spans)			X	X	X	SPANS
OpenTelemetry ingest			X	X	X	OTEL
OpenPipeline			X	X	X	OPLOGS, OPIPE
Buckets, security context, segments			X	X	X	ORGNZ
SLO definitions			X	X	X	SLO
DQL / Notebooks / Dashboards			X	X	X	DASH
Real User Monitoring			X	X	X	WEBRUM, MOBL
Application Security			X	X	X	APPSEC
Business Events				X	X	BIZEV
Dynatrace Intelligence forecasting				X	X	AIOPS
Ownership and routing by team				X	X	FAQ-21, ALERT
Cost allocation and consumption tracking				X	X	FINOPS
Workflows (AutomationEngine)					X	WFLOW
Site Reliability Guardian (release gates)					X	SLO-06
Configuration as code (Monaco / Terraform)					X	AUTOM

> **Tip:** You do not need to adopt every feature at every level. Focus on the features that address your most pressing gaps.

9. Summary and Next Steps

Key Takeaways

- Observability maturity is a spectrum from reactive (Level 1) to autonomous (Level 5)
- Assessment should be data-driven — use DQL queries to measure coverage, ingestion diversity, and alerting quality
- Not every service needs Level 5 maturity — align investment with business criticality
- Dynatrace features map directly to maturity levels, providing a clear adoption path

Next Steps

- Proceed to **ADOPT-02: Platform Health Assessment** to build a detailed scorecard of your current Dynatrace deployment
- Use the maturity assessment results to prioritize features for adoption
- Share the maturity model with leadership to align on observability investment priorities

References

- Detection of frequent issues (DT docs)
- Transition from frequent issue detection (DT docs) — *"Frequent issue detection is being phased out on the latest Dynatrace platform."*
- Primary Grail fields and tags enrichment through OneAgent (DT docs) — *"OneAgent version 1.333"*
- End of support announcements (DT docs) — *"EF1 JMX and PMI extensions reach end of support on July 1, 2027."*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.